

Maximum Likelihood Reinforcement Learning for Sparse-Reward Continuous-Control Robotics

Hemal Arora

Department of Electrical Engineering
Stanford University
hemal1@stanford.edu

Aadi Golchha

Department of Computer Science
Stanford University
aadi@stanford.edu

Abstract

We present the first application of Maximum Likelihood Reinforcement Learning (MaxRL) (Tajwar et al., 2026) to continuous-control robotics with sparse binary rewards. MaxRL reweights policy gradients by inverse success probability ($1/p$), concentrating signal on hard tasks, and has been shown to Pareto-dominate RLOO and GRPO on discrete-action domains. We apply MaxRL to `FetchPush-v4`, a 7-DOF robotic arm task, comparing against RLOO across 18 runs with varying rollout budgets $N \in \{4, 10, 20\}$ and three random seeds. Contrary to our hypothesis, RLOO consistently outperforms MaxRL. We identify four structural reasons for this negative transfer—objective–evaluation mismatch, a gradient-step budget confound, a fixed variance that removes the diversity regime where MaxRL excels, and gradient interference from large hard-goal updates—and discuss implications for extending MaxRL to continuous-control settings.

1. Introduction

Reinforcement learning in robotics with sparse binary rewards—+1 on success, 0 otherwise—is notoriously difficult. Standard policy-gradient methods (Williams, 1992) optimize the expected pass rate, weighting all tasks equally. As a consequence, tasks where the agent rarely succeeds contribute negligible gradient signal, and training stalls before meaningful improvement is observed.

Recent work by Tajwar et al. (2026) introduced Maximum Likelihood Reinforcement Learning (MaxRL), a framework that reweights gradients by inverse success probability ($1/p$), concentrating signal on hard tasks. The key observation is that the RL and ML objectives differ at the population level

by an inverse-probability reweighting factor:

$$\nabla_{\theta} J_{\text{RL}} = \mathbb{E}_x [\nabla_{\theta} p_{\theta}(x)], \quad (1)$$

$$\nabla_{\theta} J_{\text{ML}} = \mathbb{E}_x \left[\frac{1}{p_{\theta}(x)} \nabla_{\theta} p_{\theta}(x) \right]. \quad (2)$$

The $1/p_{\theta}(x)$ factor places far greater emphasis on inputs with low success probabilities—precisely those that require the most improvement. MaxRL has been shown to Pareto-dominate RLOO and GRPO on discrete action-space domains, including image classification (ImageNet), maze navigation, and LLM mathematical reasoning (GSM8K). However, MaxRL has *not* been evaluated on continuous-control robotics tasks, which differ from the settings in the original paper due to their combination of continuous high-dimensional action spaces, and physics-based dynamics.

We present the first application of MaxRL to this domain. Concretely, we train on `FetchPush-v4` (Plappert et al., 2018)—a 7-DOF robotic arm that must push a block to a randomly placed target—and compare MaxRL against RLOO across 18 runs with a varying rollout budget $N \in \{4, 10, 20\}$ and three random seeds. Contrary to expectations, RLOO consistently outperforms MaxRL. We provide a detailed analysis of four structural reasons for this negative transfer and discuss implications for applying MaxRL to continuous control.

2. Background

Problem setting. Let $x \sim \rho$ denote a goal-conditioned task. Policy π_{θ} induces a pass rate $p_{\theta}(x) = \Pr(\text{success} \mid x)$, and reward is binary: $r(\tau) = \mathbb{I}[\text{success}]$. Standard RL maximizes the expected pass rate:

$$J_{\text{RL}}(\theta) = \mathbb{E}_x [p_{\theta}(x)],$$

with gradient $\nabla_{\theta} J_{\text{RL}} = \mathbb{E}_x [\nabla_{\theta} p_{\theta}(x)]$. This weights all tasks equally, so tasks with $p_{\theta}(x) \approx 0$ contribute almost no gradient signal.

Table 1. Advantage weight functions $w(p)$ for the RL, MaxRL, and ML objectives.

	RL	MaxRL(N)	ML
$w(p)$	1	$\frac{1-(1-p)^N}{p}$	$\frac{1}{p}$

MaxRL objective. The maximum likelihood objective $J_{\text{ML}}(\theta) = \mathbb{E}_x[\log p_\theta(x)]$ admits a Maclaurin expansion in terms of fail@ k events. Defining $p := p_\theta(x)$ and $\text{fail}@k(x) := (1-p)^k$:

$$\log p = - \sum_{k=1}^{\infty} \frac{\text{fail}@k(x)}{k}. \quad (3)$$

Differentiating yields $\nabla_\theta \log p = \sum_{k=1}^{\infty} \frac{1}{k} \nabla_\theta \text{pass}@k(x)$, an infinite harmonic mixture of pass@ k gradients. Standard RL retains only the $k=1$ term. Truncating at order T gives the MaxRL family:

$$J_{\text{MaxRL}}^{(T)}(x) := - \sum_{k=1}^T \frac{(1-p)^k}{k}, \quad (4)$$

which recovers RL at $T=1$ and ML as $T \rightarrow \infty$. Crucially, because the truncation level equals the number of rollouts N , increasing compute directly improves the objective being optimized rather than merely reducing variance of a fixed objective.

Gradient estimators. The three objectives induce a family of advantage weights $w(p)$ (Table 1). As $N \rightarrow \infty$, MaxRL converges to exact ML. Figure 1 visualizes how $w(p)$ amplifies gradient signal at low success probabilities: at $p=0.05$, MaxRL($N=20$) applies $\sim 14\times$ the weight of standard RL. In practice, $p_\theta(x)$ is not available in closed form, so both MaxRL and RLOO use rollout-based advantage estimates, described in Section 3.

3. Methods

3.1. Gradient Estimators

MaxRL estimator. For task x , collect N rollouts with binary rewards $r_i \in \{0, 1\}$. Let $K = \sum_i r_i$ be the number of successes and $S_i = \nabla_\theta \log \pi_\theta(\tau_i | x)$ the score function of rollout i . The variance-reduced MaxRL estimator (Tajwar et al., 2026) assigns per-rollout advantages:

$$\hat{A}_i^{\text{MaxRL}} = \begin{cases} \frac{1}{K} - \frac{1}{N} & \text{if } r_i = 1, \\ -\frac{1}{N} & \text{if } r_i = 0, \end{cases}$$

with no update when $K=0$. The gradient $\hat{g}(x) = \sum_i \hat{A}_i S_i$ is unbiased for $\nabla_\theta J_{\text{MaxRL}}^{(N)}$, so increasing N improves the objective itself, not just the variance.

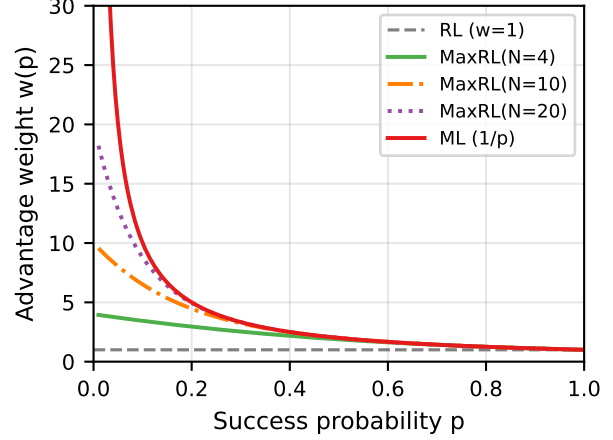


Figure 1. Advantage weight $w(p)$ as a function of success probability. RL weights all tasks equally ($w=1$). MaxRL interpolates between RL and ML ($1/p$), with higher N approaching the full inverse-probability reweighting.

RLOO baseline. REINFORCE leave-one-out (RLOO) (Ahmadian et al., 2024) is unbiased for the pass@1 gradient $\nabla_\theta p_\theta(x)$, with advantages:

$$\hat{A}_i^{\text{RLOO}} = \begin{cases} \frac{N-K}{N(N-1)} & \text{if } r_i = 1, \\ \frac{-K}{N(N-1)} & \text{if } r_i = 0. \end{cases}$$

Increasing N under RLOO reduces variance of a *fixed* objective (pass@1) but does not improve the optimization target. This is the key theoretical distinction: more rollouts help MaxRL optimize a *better* objective, while they only help RLOO optimize the *same* objective more precisely.

3.2. GCBC Pretraining

Both methods start from a shared checkpoint trained via Goal-Conditioned Behavior Cloning (GCBC) with Hind-sight Experience Replay (HER) (Andrychowicz et al., 2018). A warm start is critical because both MaxRL and RLOO produce zero gradient when $K = 0$ for a goal. We gather 5,000 episodes via a 50/50 mix of random actions and a reach-toward-block controller, filter for trajectories that achieve a minimum block displacement threshold (≥ 2 cm), and relabel via HER ($k = 4$ future goals per timestep). This allows us to relabel unsuccessful trajectories as successful ones for the *achieved* block displacement, warm starting the learning process. The policy trains for 30 epochs, reaching $\sim 18\%$ success rate.

3.3. RL Fine-Tuning

From the GCBC checkpoint, we fine-tune with MaxRL and RLOO using Adam (Kingma & Ba, 2017). Table 2

Table 2. Shared hyperparameters for RL fine-tuning.

HYPERPARAMETER	VALUE
LEARNING RATE (ADAM)	10^{-4}
OUTPUT STD. DEV. σ	0.3 (FIXED)
GOALS PER ITERATION	20
EVAL GOALS (GREEDY)	50
EVAL ROLLOUTS PER GOAL	20
ROLLOUT BUDGET N	{4, 10, 20}
RANDOM SEEDS	{3, 5, 10}
TOTAL ENV. STEPS	10M

Algorithm 1 MaxRL / RLOO Fine-Tuning

Input: GCBC checkpoint θ_0 , rollout budget N , goals per iteration $G=20$

for iteration $t = 1, 2, \dots$ **do**

 Sample G goals $\{x_j\}_{j=1}^G \sim \rho$

for each goal x_j **do**

 Collect N rollouts $\{\tau_i\}_{i=1}^N$ with π_θ

 Compute $K_j = \sum_i r_i$ (number of successes)

 Compute advantages $\hat{A}_i$ via MaxRL or RLOO

 Accumulate $\hat{g}_j = \sum_i \hat{A}_i \nabla_\theta \log \pi_\theta(\tau_i | x_j)$

end for

 Update $\theta \leftarrow \theta - \alpha \cdot \frac{1}{G} \sum_j \hat{g}_j$ (Adam)

if $t \bmod \text{eval_freq} = 0$ **then**

 Evaluate on 50 goals using deterministic mean

end if

end for

summarizes the shared hyperparameters. We vary $N \in \{4, 10, 20\}$ across seeds $\{3, 5, 10\}$, yielding 18 total runs. Only the advantage computation differs between the two methods.

3.4. Policy Architecture

We use a two-hidden-layer MLP (256 units per layer, ReLU) with a Gaussian policy: the network outputs the mean action vector (one dimension per DOF; 7 for the Fetch robot), and the standard deviation is fixed ($\sigma = 0.3$). Observations are normalized using running mean and variance loaded from the GCBC checkpoint. Deterministic evaluation uses the policy mean, which is standard practice in robotics benchmarks.

3.5. Training Procedure

Algorithm 1 summarizes the full training loop shared by both methods. The only difference between MaxRL and RLOO is the advantage computation in line 7.

Table 3. Final success rate (%) by method and rollout budget N (mean $\pm$ SE, 3 seeds). Gradient steps decrease as N increases under a fixed 10M step budget.

	$N=4$	$N=10$	$N=20$
GRAD. STEPS	2,500	1,000	500
RLOO	91.3	88.6	82.0
MAXRL	90.0	85.0	76.0
Δ (RLOO – MAXRL)	+1.3	+3.6	+6.0

4. Experiments

Learning curves. Figure 2 shows mean $\pm$ SE success rate across 3 seeds for each (N , method) combination. Table 3 summarizes the final success rates. At $N=4$ both methods reach approximately 90%, with RLOO leading slightly. The gap widens at $N=10$ (RLOO 88.6% vs. MaxRL 85%) and grows further at $N=20$ (RLOO 82% vs. MaxRL 76%). Across all 18 runs, **RLOO consistently outperforms MaxRL.**

Compute scaling. Figure 5 plots final success rate as a function of N . Contrary to the compute-scaling prediction from Tajwar et al. (2026), both methods *decline* with larger N . MaxRL degrades more steeply (90% $\rightarrow$ 76%) than RLOO (91.3% $\rightarrow$ 82%). This result is confounded: to fit our compute budget, we fixed total environment steps at 10M, so increasing N reduces the number of gradient steps (2,500 at $N=4$ vs. 500 at $N=20$).

Pass@ k evaluation. Figure 3 shows pass@ k over training at $N=20$ evaluation rollouts. At pass@1, RLOO dominates (80% vs. 70%). At $k \geq 4$, both methods converge to near-identical coverage (~ 98 –100%). Neither method exhibits pass@ k collapse, which is expected: the fixed $\sigma=0.3$ prevents the policy from reducing its own output diversity, unlike LLM settings where token entropy can shrink freely.

Gradient norm analysis. Figure 4 plots gradient norm against per-task success fraction K/N , pooling all seeds and N values. MaxRL concentrates gradient on hard goals (low K/N), producing norms of 200–300 at $K/N < 0.1$, consistent with the theoretical $1/p$ reweighting. RLOO distributes signal uniformly at 20–40 across all K/N values. Despite matching theoretical predictions, MaxRL’s hard-goal emphasis does not translate to better performance.

5. Analysis

Our results constitute a **negative transfer** finding. The MaxRL advantage weights transfer correctly to the continuous-control domain—the $1/p$ reweighting behaves as predicted (Figure 4)—yet this reweighting does not im-

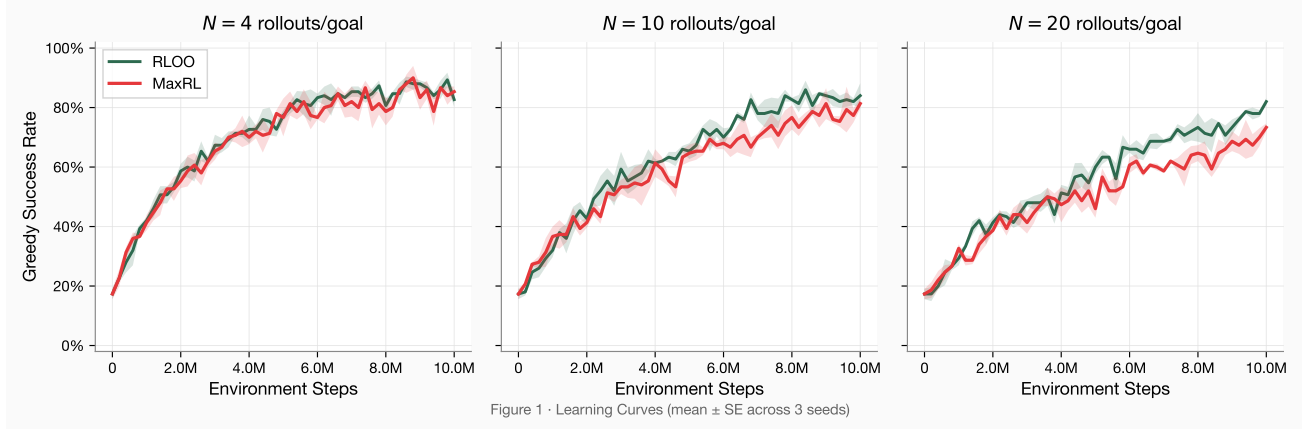


Figure 2. Learning curves (mean $\pm$ SE, 3 seeds) for MaxRL and RLOO at $N \in \{4, 10, 20\}$. RLOO leads at all rollout budgets, with the gap widening as N increases.

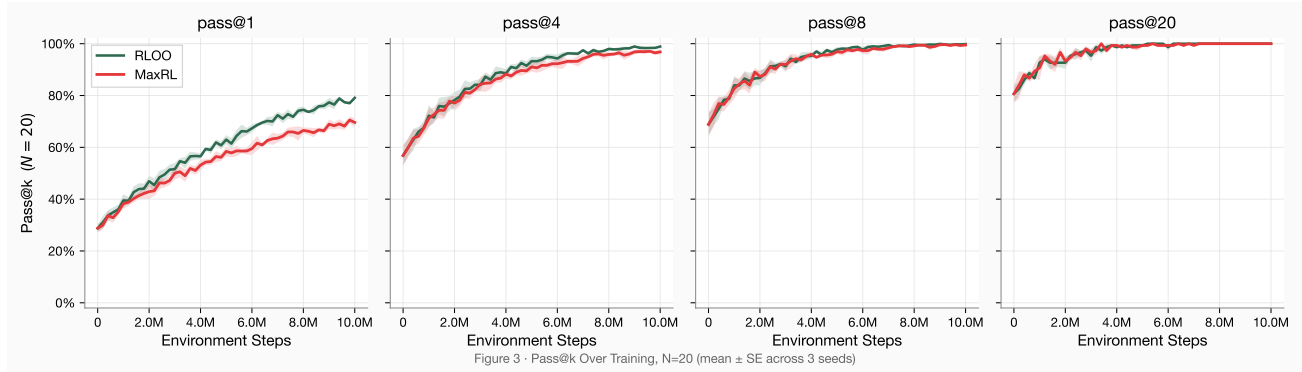


Figure 3. Pass@ k over training ($N=20$ evaluation rollouts). RLOO leads at pass@1; both converge to $\sim 100\%$ at $k \geq 4$. Neither method shows pass@ k collapse.

prove final performance. We identify four structural reasons.

1. Objective–evaluation mismatch. Evaluation uses the deterministic policy mean, which is standard in robotics. RLOO optimizes expected pass rate, directly aligned with greedy success. MaxRL optimizes toward $\log p_\theta(x)$, up-weighting hard goals at the expense of easy ones. This can *hurt* greedy performance on already-solved goals. By contrast, the experiments in [Tajwar et al. \(2026\)](#) evaluate via stochastic sampling (pass@ k , majority vote), which aligns more naturally with MaxRL’s objective. Evaluating our policies stochastically would more directly test whether MaxRL’s optimization target is being achieved.

2. Gradient steps confounded with rollout budget. Our experimental design holds total environment steps constant due to compute constraints. As a result, $N=20$ runs receive only 500 gradient updates vs. 2,500 for $N=4$ runs. MaxRL’s

benefit—a higher-order ML approximation—requires sufficient updates to exploit a better objective. The maze scaling experiments in [Tajwar et al. \(2026\)](#) use 9,000 gradient steps even at 128 rollouts per task, suggesting the improvement from a higher-order objective requires a far more generous update budget.

3. Fixed variance masks the diversity lever. With learnable σ , standard RL methods are expected to gradually sharpen the policy distribution (reducing pass@ k), while MaxRL’s $1/p$ reweighting should resist this collapse—the core advantage demonstrated by [Tajwar et al. \(2026\)](#). However, with learnable σ , MaxRL diverged in our experiments: the $1/K$ reweighting creates a feedback loop in which shrinking σ reduces K , which amplifies the $1/K$ term further. Fixing $\sigma=0.3$ stabilized training but eliminates the diversity regime in which MaxRL is strongest. Figure 3 confirms this: neither method shows any pass@ k collapse, so

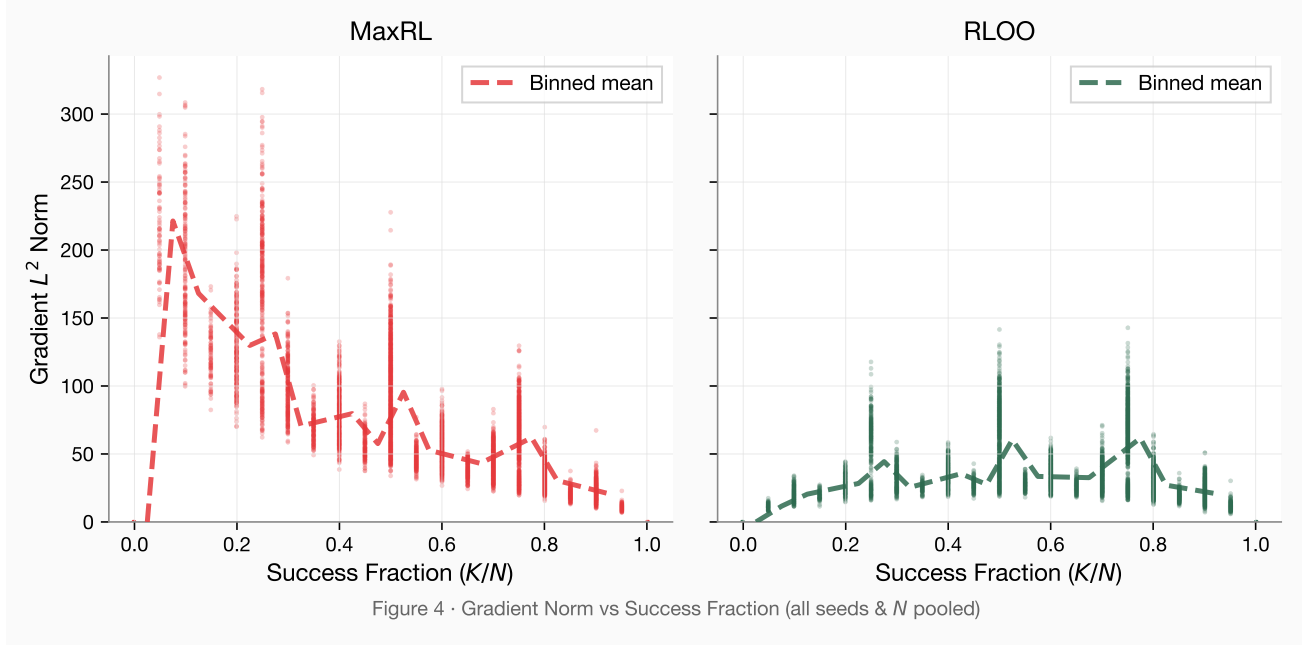


Figure 4. Gradient norm vs. per-task success fraction K/N (all seeds and N pooled). MaxRL concentrates large gradients on hard goals as theory predicts; RLOO is near-flat.

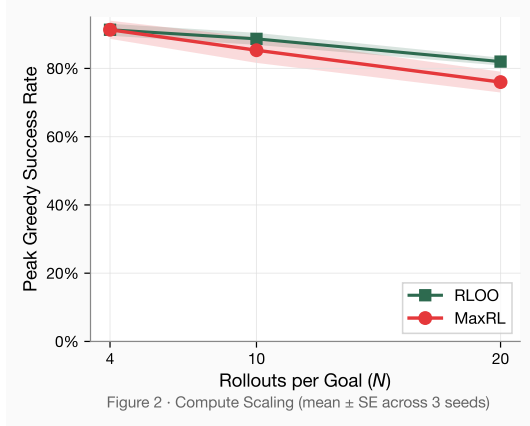


Figure 5. Final success rate vs. rollout budget N . Both methods decline as N increases under a fixed total environment step budget, with MaxRL degrading more steeply.

MaxRL’s primary demonstrated advantage is not operative.

4. Hard-goal gradient interference. Figure 4 shows MaxRL producing gradient norms of 200–300 at $K/N < 0.1$, compared to RLOO’s flat 20–40 across all success fractions. In a 256-256 MLP with shared representations across goal difficulties, these outsized updates from hard goals overwrite representations useful for easier goals, producing a net performance decline.

6. Conclusions

Across all 18 runs, RLOO consistently outperforms MaxRL on `FetchPush-v4`. MaxRL’s gradient reweighting transfers to the continuous-control domain in the sense that hard goals receive stronger gradients as theory predicts (Figure 4). However, this does not translate to better performance under deterministic evaluation. Moreover, MaxRL’s $1/K$ reweighting caused divergence with learnable output variance, requiring a fixed σ that removes the diversity regime in which MaxRL’s advantages are strongest.

Our results suggest MaxRL’s benefits are domain-specific, benefiting most from (a) stochastic evaluation metrics such as `pass@k` or majority vote, (b) settings where preserving output diversity is a live concern, and (c) sufficient gradient steps to exploit the higher-order objective. All three conditions held in the original paper’s experiments and none fully held in ours. Future work should evaluate MaxRL in continuous control with an equal gradient-step budget across rollout configurations, a stabilized adaptive variance mechanism, and stochastic evaluation metrics.

Group Contributions We both contributed to this project equally. Aadi was primarily responsible for the coding portion of the project, and Hemal was primarily responsible for the write-up, although we both contributed to all areas of the project.

Github Link https://github.com/hemalarora1/cs234_project.git

AI Use Disclosure. Claude was used to assist with text edits and coding, with human review throughout.

References

- Ahmadian, A., Cremer, C., Gallé, M., Fadaee, M., Kreutzer, J., Pietquin, O., Üstün, A., and Hooker, S. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms, 2024. URL <https://arxiv.org/abs/2402.14740>.
- Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Abbeel, P., and Zaremba, W. Hindsight experience replay, 2018. URL <https://arxiv.org/abs/1707.01495>.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization, 2017. URL <https://arxiv.org/abs/1412.6980>.
- Plappert, M., Andrychowicz, M., Ray, A., McGrew, B., Baker, B., Powell, G., Schneider, J., Tobin, J., Chociej, M., Welinder, P., Kumar, V., and Zaremba, W. Multi-goal reinforcement learning: Challenging robotics environments and request for research, 2018. URL <https://arxiv.org/abs/1802.09464>.
- Tajwar, F., Zeng, G., Zhou, Y., Song, Y., Arora, D., Jiang, Y., Schneider, J., Salakhutdinov, R., Feng, H., and Zanette, A. Maximum likelihood reinforcement learning, 2026. URL <https://arxiv.org/abs/2602.02710>.
- Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, 1992. doi: 10.1007/BF00992696.